Lista zadań nr 2 19, 21 października 2026 r.

zaawansowane techniki C++ i STL

inteligentne wskaźniki

Instytut Informatyki
Uniwersytetu Wrocławskiego **Paweł Rzechonek**

Zadanie 1 (3 pkt)

Zdefiniuj prostą klasę zawierającą licznik typu `uint64_t`, początkowo ustawiony na wartość 1. W kasie tej, oprócz innych funkcjonalności, umieść wirtualny destruktor, który wypisze komunikat ze stanem licznika na standardowym wyjściu dla błędów `cerr`. Następnie stwórz n-elementową tablicę takich liczników i opakuj ją wskaźnikiem `unique_ptr` (opakowanie ma dotyczyć tablicy a nie pojedynczych liczników). Napisz też funkcję, która wywoła się rekurencyjnie m razy z argumentem będącym wskaźnikiem na taką tablicę (wskaźnik `unique_ptr` ustaw za pomocą przeniesienia). W i-tym wywołaniu funkcja ta ma zwiększyć o wartość i 10% losowo wybranych elementów.

W programie głównym wywołaj tą funkcję, a po jej wywołaniu wypisz wartości wszystkich n liczników umieszczonych w tablicy. Parametry n i m należy ustalić arbitralnie (na przykład n = 10 i m = 10).

Zadanie 2 (3 pkt)

Zdefiniuj klasę opakowującą plik tekstowy `line_writer`. Obiekt takiej klasy powinien być opakowaniem dla strumienia plikowego `ofstream` otwartego w konstruktorze. Zadaniem obiektów tej klasy będzie pisanie do pliku wiersz po wierszu. Klasa ta powinna zamykać strumień w destruktorze.

W programie głównym stwórz kilka wskaźników `shared_ptr` odnoszących się do tego samego obiektu plikowego. Niech każdy fragment programu posiadający taki własny wskaźnik zapisze w tym pliku swoje dane. Plik powinien zostać zamknięty dopiero, gdy wszystkie sprytne wskaźniki zostaną zlikwidowane.

Zadanie 3 (4 pkt)

Zdefiniuj klasę `llama` reprezentującą pojedynczą lamę w stadzie lam (zarówno w konstruktorze jak i w destruktorze wypisuj spersonalizowane komunikaty dotyczące lamy). W obiekcie tym powinno być zapisane imię lamy typu `string` (imiona niech będą unikatowe w stadzie), jej płeć jako typ wyliczeniowy, wskaźnik na ojca i matkę typu `shared_ptr<llama>` (dopuszczamy przypadek nieznanego ojca lub matki oznaczonego za pomocą statycznego obiektu `noname`) oraz wskaźnik na kolekcję dzieci typu `vector<weak_ptr<llama>>`.

Stado lam reprezentowane przez klasę `herd` niech będzie kolekcją obiektów `llama` – implementując tą klasę można wykorzystać jakąś popularną kolekcję standardową, na przykład `set<llama>` (porównywanie lam według imion). Przeprowadź symulację życia lam w stadzie: hodowca może dokupić lamę z innego stada (wtedy jej rodzice są nieznani), lama może się urodzić w stadzie (wtedy jej rodzice są członkami stada, ale ojciec może być nieznany) i wreszcie lama może zostać sprzedana (należy ją wtedy usunąć z kolekcji). Zaobserwuj, co się stanie, gdy słaby wskaźnik `weak_ptr` stanie się wskaźnikiem wiszącym (po zwolnieniu zasobu przez wskaźnik macierzysty `shared_ptr`).